

Module 7

Real-World Applications

Learning objectives

- Apply UVM to real-world verification scenarios

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["DMA Verification"]

T2["Protocol Verification"]

T1 --> T2

T3["VIP Development"]

T2 --> T3

Apply UVM to real-world verification scenarios

Overview

- This module applies UVM to real-world verification scenarios including DMA verification, protocol v...

Design architecture

1. Real-world verification architecture (1/3)

- DMA path: CPU/config →
descriptor ring → bus master →
memory/peripheral
- Protocol path: serial frame →
byte stream → transaction →
scoreboard
- VIP encapsulates protocol
knowledge for reuse across
projects and modules
- `module7/dut/dma/simple\_dma.v`
and `dut/protocols/uart.v`
anchor the labs

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

A["Add rtl_architecture.mmd"] --> B["for Module 7"]

1. Real-world verification architecture (2/3)

- DMA agent: Programs descriptors; monitors transfer completion
- Protocol agents (UART/SPI/I2C): Bus-specific driver/monitor/sequencer sets
- VIP package: Reusable agent + sequences + config for a protocol
- Scoreboard / predictor: End-to-end data integrity across DMA + peripheral

1. Real-world verification architecture (3/3)

- Reference model: Software model of DMA or protocol behavior

2. VIP and reuse architecture (1/2)

- 1: VIP interface — Standardized signal bundle + modport
- 2: VIP agent — Configurable active/passive agent
- 3: VIP sequences — Library of legal and stress sequences
- 4: Integration env —
Instantiates DUT + VIP + scoreboard

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR

A["Add verification_architecture.mmd"] --> B["for Module 7"]

2. VIP and reuse architecture (2/2)

- 5: Test suite — Directed + random tests using VIP API

3. Module 7 application map (1/2)

- `examples/dma/` — descriptor programming, transfer status, interrupts
- `examples/protocols/` — UART, SPI, I2C patterns (timing and framing)
- `examples/vip/` — packaging reusable verification IP
- `examples/best\_practices/` — naming, config, debug, regression discipline

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 7"]

3. Module 7 application map (2/2)

- ``test_real_world_uvm.sv`` — integrated scenario combining subsystems

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

A["Add rtl_architecture.mmd"] --> B["for Module 7"]

Module 7: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

A["Add verification_architecture.mmd"] --> B["for Module 7"]

Module 7: stimulus, observation, and checking.

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module7 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Artifact flow (spec → sim) (1)

- 1: VIP interface — Standardized signal bundle + modport
- 2: VIP agent — Configurable active/passive agent
- 3: VIP sequences — Library of legal and stress sequences
- 4: Integration env — Instantiates DUT + VIP + scoreboard

Artifact flow (spec → sim) (2)

- 5: Test suite — Directed + random tests using VIP API

Example directory layout

- DMA agent: Programs descriptors; monitors transfer completion
- Protocol agents (UART/SPI/I2C): Bus-specific driver/monitor/sequencer sets
- VIP package: Reusable agent + sequences + config for a protocol
- Scoreboard / predictor: End-to-end data integrity across DMA + peripheral
- Reference model: Software model of DMA or protocol behavior

Directed test execution sequence

- VIP config
- env build
- sequence library
- coverage + report

Verification & testing methods

1. Application-specific verification strategy

- DMA: descriptor alignment, boundary lengths, error flags, concurrent channels
- Serial protocols: baud/bit timing, framing, parity, back-to-back packets
- VIP usage: configure via object; run standard sequences before custom stress
- Directed tests prove basic paths; random tests explore corner combinations

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 7"]

2. Production best practices (1/2)

- Centralize plusargs via CLP
(Module 8) for seed, verbosity,
test name
- Separate test intent (test
class) from reusable env (VIP +
env)
- Log protocol transactions at
UVM_MEDIUM; errors at UVM_LOW
- Regression:
`./scripts/module7.sh --all-
examples --uvm-tests`

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 7"]

2. Production best practices (2/2)

- Closure: functional coverage bins hit + scoreboard clean + VIP protocol clean

Syllabus topics

Protocol & design details

Detail: Example 7.1: DMA Verification (`module7/examples/dma/dma.sv`)

- Interface (`dma\_if`): DMA interface with start, done, addresses, length, and channel signals
- Transaction (`DmaTxn`): DMA transaction with source address, destination address, length, and chann...
- Sequence (`DmaSeq`): DMA sequence generating random DMA transfers
- Driver (`DmaDriver`): DMA driver implementing DMA protocol

Detail: Example 7.1: DMA Verification (`module7/examples/dma/dma.sv`)...

- Environment (`DmaEnv`): DMA verification environment
- Test (`DmaTest`): DMA verification test with simple DMA DUT
- Method Signature: `function bit randomize();`
- Purpose: Generate random DMA transfer parameters

Detail: Example 7.2: UART Verification (`module7/examples/protocols/u...

- Interface (`uart\_if`): UART interface with TX/RX signals and control
- Transaction (`UartTxn`): UART transaction with data field
- Sequence (`UartSeq`): UART sequence generating random UART transfers
- Driver (`UartDriver`): UART driver implementing UART protocol

Detail: Example 7.2: UART Verification (`module7/examples/protocols/u...

- Environment (`UartEnv`): UART verification environment
- Test (`UartTest`): UART verification test with UART DUT
- Method Signature:
- `task get\_next\_item(ref T item);` - Get transaction from sequencer

Detail: Example 7.3: SPI Verification (`module7/examples/protocols/sp...

- Test (`SpiExampleTest`): SPI verification test scaffold
- UVM Pattern: Demonstrates UVM structure for SPI verification
- SPI protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development

Detail: Example 7.3: SPI Verification (`module7/examples/protocols/sp...

- Protocol-specific testbench structure supports future DUT integration

Detail: Example 7.4: I2C Verification (`module7/examples/protocols/i2...

- Test (`I2cExampleTest`): I2C verification test scaffold
- UVM Pattern: Demonstrates UVM structure for I2C verification
- I2C protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development

Detail: Example 7.4: I2C Verification (`module7/examples/protocols/i2...

- Protocol-specific testbench structure supports future DUT integration

Detail: Example 7.5: VIP Development (`module7/examples/vip/vip.sv`)...

- Transaction (`VipTxn`): VIP transaction with payload
- Sequence (`VipSeq`): VIP sequence generating transactions
- Driver (`VipDriver`): VIP driver
- Agent (`VipAgent`): VIP agent containing sequencer and driver

Detail: Example 7.5: VIP Development (``module7/examples/vip/vip.sv``)...

- Test (``VipTest``): VIP test demonstrating VIP usage
- Purpose: Package VIP components (sequencer + driver) in agent
- Pattern: Agent encapsulates all VIP components
- Key: Standard UVM agent structure for reusability

Detail: Example 7.6: Best Practices (`module7/examples/best_practices...

- Test (`BestPracticesTest`): Test demonstrating best practices
- Phase Organization: Proper phase usage and logging
- Topology Printing: Component hierarchy visualization
- Logging Patterns: Structured logging for maintainability

Detail: Example 7.6: Best Practices (`module7/examples/best_practices...

- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging
- Method Signature:

Detail: DMA Transaction Randomization (1)

- Method Signature: ``function bit randomize();``
- Return Type: ``bit`` (1 = success, 0 = failure)
- Purpose: Generate random DMA transfer parameters
- Usage: ``void'(t.randomize() with { len inside
{[1:20]}; });``

Detail: DMA Transaction Randomization (2)

- Parameters: None (randomizes all `rand` fields in object)
- Constraints: Use `with { constraint\_expression; }` to limit values
- Example: `len inside {[1:20]}` constrains transfer length to 1-20 bytes
- Key: `void`() suppresses Verilator warning about ignored return value

Detail: DMA Protocol Implementation

- Pattern: Start pulse → Wait for done → Completion
- Key: `do-while` loop waits for `dma\_done` pulse

Detail: UART Protocol Implementation (1)

- Pattern: Data → Start → Wait for busy deassert → Completion
- Key: Wait for `tx\_busy` to deassert
- Event Control: `@(posedge clk)` - wait for positive clock edge
- Non-blocking Assignment: `<==` - schedule assignment for next time step

Detail: UART Protocol Implementation (2)

- Do-While Loop: ``do @(posedge clk); while (condition);`` - wait for condition
- Purpose: Enable loopback testing
- Use Case: Minimal smoke testing without external device

Detail: VIP Agent Packaging

- Purpose: Package VIP components for reusability
- Pattern: Agent encapsulates sequencer + driver
- Key: Standard UVM agent structure

Detail: VIP Logging (1)

- Macro: ``uvm_info(id, message, verbosity)``
- Function: ``$sformatf(format, args)`` → string
- Purpose: Log transaction with formatted message
- Format Specifiers:

Detail: VIP Logging (2)

- ``%02h`` - 2-digit hexadecimal, zero-padded
- ``%08h`` - 8-digit hexadecimal, zero-padded
- ``%0d`` - Decimal, no padding
- ``%s`` - String

Detail: Phase-Based Logging

- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging

Detail: Topology Printing (1)

- Method Signature:
- ``static function uvm_root get();`` → returns singleton root
- ``function void print_topology();`` → prints hierarchy
- Purpose: Visualize component hierarchy

Detail: Topology Printing (2)

- Timing: Call in ``end_of_elaboration_phase()` (after all connections)
- Output: Prints complete component tree with hierarchical paths
- Usage:

Detail: end_of_elaboration_phase (1)

- Phase Type: Build-time phase (function, executes bottom-up)
- Purpose: Final elaboration tasks after all connections
- Timing: After ``connect_phase()`, before ``start_of_simulation_phase()`
- Common Uses:

Detail: end_of_elaboration_phase (2)

- Print topology
- Validate configuration
- Final checks
- Setup verification

Verification flow (reference)

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 7"]

Stimulus, DUT response, and check points for this module.

1. DMA Verification (1/9)

- DMA Controller Verification
- DMA transaction modeling
- DMA protocol implementation
- DMA sequence generation
- DMA result checking
- DMA Patterns

1. DMA Verification (2/9)

- DMA transfer patterns
- Multi-channel DMA
- DMA completion handling
- DMA error handling
- Interface (``dma_if``): DMA interface with start, done, addresses, length, and channel signals
- Transaction (``DmaTxn``): DMA transaction with source address, destination address, length, and chann...

1. DMA Verification (3/9)

- Sequence (``DmaSeq``): DMA sequence generating random DMA transfers
- Driver (``DmaDriver``): DMA driver implementing DMA protocol
- Environment (``DmaEnv``): DMA verification environment
- Test (``DmaTest``): DMA verification test with simple DMA DUT
- Method Signature: ``function bit randomize();``
- Purpose: Generate random DMA transfer parameters

1. DMA Verification (4/9)

- Constraints: Use ``with { constraint; }`` to limit values
- Key: ``len inside {[1:20]}`` constrains transfer length
- Method Signature:
- ``task get_next_item(ref T item);`` - Get transaction from sequencer
- ``task item_done();`` - Signal transaction completion
- Purpose: Implement DMA protocol with start/done handshake

1. DMA Verification (5/9)

- Protocol Flow:
- Key: ``do-while`` loop waits for ``dma_done`` pulse (blocking wait)
- Timing: Start signal is one clock cycle pulse
- Blocking Behavior: ``get_next_item()`` blocks until sequencer provides transaction, ``item_done()`` unb...
- Completion Detection: Wait for ``dma_done`` signal assertion (may take multiple clock cycles)
- Method Signature: ``task body();`` (no parameters, no return)

1. DMA Verification (6/9)

- Purpose: Generate and send multiple DMA transfers to sequencer
- Execution Flow:
- Key: ``repeat`` loop creates multiple transactions sequentially
- Blocking Behavior:
- ``start_item()`` blocks until sequencer grants access
- ``finish_item()`` blocks until driver completes transaction (may take many cycles)

1. DMA Verification (7/9)

- Transaction Lifetime:
- Created in loop
- Sent to sequencer via ``start_item()``
- Driver processes it (drives signals, waits for DMA done)
- Driver signals completion via ``item_done()``
- ``finish_item()`` returns, loop continues

1. DMA Verification (8/9)

- Constraint: `len inside {[1:20]}` ensures reasonable transfer lengths
- DMA transaction modeling captures transfer semantics
- DMA protocol implementation handles start/done handshake
- DMA sequence generation creates realistic transfer scenarios
- DMA result checking verifies transfer completion
- DMA Protocol: Start pulse → Wait for done → Completion

1. DMA Verification (9/9)

- Transaction Flow: Sequence → Sequencer → Driver → DUT
- Completion Detection: Wait for `dma\_done` signal assertion
- DMA transaction generation
- DMA protocol execution
- DMA transfer completion
- UVM report summary

2. Protocol Verification (1/9)

- UART Verification
- UART protocol modeling
- Serial communication verification
- Loopback testing
- UART-specific patterns
- SPI Verification

2. Protocol Verification (2/9)

- SPI protocol verification patterns
- UVM scaffold for SPI
- Protocol-specific testbench structure
- I2C Verification
- I2C protocol verification patterns
- UVM scaffold for I2C

2. Protocol Verification (3/9)

- Protocol-specific testbench structure
- Interface (``uart_if``): UART interface with TX/RX signals and control
- Transaction (``UartTxn``): UART transaction with data field
- Sequence (``UartSeq``): UART sequence generating random UART transfers
- Driver (``UartDriver``): UART driver implementing UART protocol
- Environment (``UartEnv``): UART verification environment

2. Protocol Verification (4/9)

- Test (``UartTest``): UART verification test with UART DUT
- Method Signature:
- ``task get_next_item(ref T item);`` - Get transaction from sequencer
- ``task item_done();`` - Signal transaction completion
- Purpose: Implement UART TX protocol
- Protocol Flow:

2. Protocol Verification (5/9)

- Key: Wait for ``tx_busy`` to deassert (transmission complete)
- Timing: Start signal is one clock cycle pulse
- Blocking Behavior: ``get_next_item()`` blocks until sequencer provides transaction, ``item_done()`` unb...
- Completion Detection: Wait for ``tx_busy`` signal deassertion (may take many clock cycles for serial...)
- UART Characteristics: Serial transmission takes time (one bit per baud period)
- Purpose: Enable loopback testing

2. Protocol Verification (6/9)

- Pattern: ``assign rx = tx`` creates loopback path
- Use Case: Minimal smoke testing without external device
- UART protocol modeling captures serial communication
- Serial communication verification tests TX/RX paths
- Loopback testing enables minimal smoke testing
- Protocol-specific patterns provide reusable verification components

2. Protocol Verification (7/9)

- UART Protocol: Data → Start → Wait for busy → Completion
- Loopback Pattern: TX output connected to RX input
- Busy Signal: Wait for `tx\_busy` deassertion
- UART transaction generation
- UART protocol execution
- UART TX/RX operation

2. Protocol Verification (8/9)

- Loopback verification
- Test (`SpiExampleTest`): SPI verification test scaffold
- UVM Pattern: Demonstrates UVM structure for SPI verification
- SPI protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development
- Protocol-specific testbench structure supports future DUT integration

2. Protocol Verification (9/9)

- Test (``I2cExampleTest``): I2C verification test scaffold
- UVM Pattern: Demonstrates UVM structure for I2C verification
- I2C protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development
- Protocol-specific testbench structure supports future DUT integration

3. VIP Development (1/6)

- VIP Architecture
- VIP component organization
- VIP packaging patterns
- VIP reuse strategies
- VIP configuration
- VIP Components

3. VIP Development (2/6)

- Transaction classes
- Sequence classes
- Driver and monitor
- Agent packaging
- Transaction (``VipTxn``): VIP transaction with payload
- Sequence (``VipSeq``): VIP sequence generating transactions

3. VIP Development (3/6)

- Driver (``VipDriver``): VIP driver
- Agent (``VipAgent``): VIP agent containing sequencer and driver
- Test (``VipTest``): VIP test demonstrating VIP usage
- Purpose: Package VIP components (sequencer + driver) in agent
- Pattern: Agent encapsulates all VIP components
- Key: Standard UVM agent structure for reusability

3. VIP Development (4/6)

- Purpose: VIP driver with structured logging
- Logging: Use ``uvm_info`` with formatted messages
- Key: ``$sformatf()`` for formatted string generation
- Purpose: Demonstrate VIP usage in test
- Pattern: Create VIP agent, use its sequencer
- Key: VIP agent is reusable component

3. VIP Development (5/6)

- VIP architecture enables reusable verification components
- VIP component organization provides clear structure
- VIP packaging patterns support easy integration
- VIP reuse strategies improve productivity
- VIP Packaging: Transaction + Sequence + Driver + Agent
- Reusability: VIP agent can be instantiated in any test

3. VIP Development (6/6)

- Structured Logging: Use ``uvm_info`` with formatted messages
- VIP transaction generation
- VIP agent operation
- VIP packaging demonstration
- VIP reuse pattern

4. Best Practices (1/10)

- Code Organization
- Component organization
- File structure
- Module organization
- Package organization
- Naming Conventions

4. Best Practices (2/10)

- Component naming
- Variable naming
- File naming
- Test naming
- Logging and Debugging
- Structured logging

4. Best Practices (3/10)

- Phase-based logging
- Topology visualization
- Debug techniques
- Test (`BestPracticesTest`): Test demonstrating best practices
- Phase Organization: Proper phase usage and logging
- Topology Printing: Component hierarchy visualization

4. Best Practices (4/10)

- Logging Patterns: Structured logging for maintainability
- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging
- Method Signature:
- ``static function uvm_root get();`` → returns singleton root instance

4. Best Practices (5/10)

- ``function void print_topology();`` → prints hierarchy
- Purpose: Visualize component hierarchy for debugging
- Timing: Call in ``end_of_elaboration_phase()`` (after all connections)
- Output: Prints complete component tree with hierarchical paths
- Output Format:
- Key: Helps debug component structure and connections

4. Best Practices (6/10)

- When to Use:
- After all components created
- After all connections made
- When debugging component hierarchy issues
- To verify testbench structure
- Benefits:

4. Best Practices (7/10)

- Visual representation of testbench structure
- Helps identify missing components
- Verifies hierarchical naming
- Aids in debugging connection issues
- Method Signature: ``function void
 end_of_elaboration_phase(uvm_phase phase);``
- Phase Type: Build-time phase (function, executes bottom-up)

4. Best Practices (8/10)

- Purpose: Final elaboration tasks after all connections
- Common Uses: Print topology, validate configuration, final checks
- Timing: After ``connect_phase()``, before ``start_of_simulation_phase()``
- Code organization improves maintainability
- Naming conventions enhance readability
- Phase-based logging provides clear execution flow

4. Best Practices (9/10)

- Topology visualization aids debugging
- Phase Logging: Log at start of each phase
- Topology Printing: Use ``print_topology()`` in ``end_of_elaboration_phase()``
- Structured Logging: Use consistent message IDs and formats
- `end_of_elaboration_phase`: Use for final checks and topology printing
- Phase-based logging

4. Best Practices (10/10)

- Component topology printing
- Best practices demonstration
- Structured logging output

5. System-Level Verification (1/24)

- Multi-DUT Verification
- Multiple DUT instantiation
- Multi-DUT coordination
- System integration
- System-level testing
- Real-World Patterns

5. System-Level Verification (2/24)

- Practical verification scenarios
- Production-quality testbenches
- Maintainable verification environments
- Scalable verification solutions
- Method Signature: ``function bit randomize();``
- Return Type: ``bit`` (1 = success, 0 = failure)

5. System-Level Verification (3/24)

- Purpose: Generate random DMA transfer parameters
- Usage: ``void'(t.randomize() with { len inside {[1:20]}; });``
- Parameters: None (randomizes all ``rand`` fields in object)
- Constraints: Use ``with { constraint_expression; }`` to limit values
- Example: ``len inside {[1:20]}`` constrains transfer length to 1-20 bytes
- Key: ``void'()` suppresses Verilator warning about ignored return value

5. System-Level Verification (4/24)

- Constraint Syntax:
- ``len inside {[1:20]}`` - Range constraint
- ``len inside {1, 2, 4, 8, 16}`` - Set constraint
- ``len > 0 && len < 100`` - Boolean constraint
- Randomization Order:
- Pattern: Start pulse → Wait for done → Completion

5. System-Level Verification (5/24)

- Key: ``do-while`` loop waits for ``dma_done`` pulse
- Pattern: Data → Start → Wait for busy deassert → Completion
- Key: Wait for ``tx_busy`` to deassert
- Event Control: ``@(posedge clk)`` - wait for positive clock edge
- Non-blocking Assignment: ``<=`` - schedule assignment for next time step
- Do-While Loop: ``do @(posedge clk); while (condition);`` - wait for condition

5. System-Level Verification (6/24)

- Purpose: Enable loopback testing
- Use Case: Minimal smoke testing without external device
- Purpose: Package VIP components for reusability
- Pattern: Agent encapsulates sequencer + driver
- Key: Standard UVM agent structure
- Macro: ``uvm_info(id, message, verbosity)``

5. System-Level Verification (7/24)

- Function: ``$sformatf(format, args)`` → string
- Purpose: Log transaction with formatted message
- Format Specifiers:
 - ``%02h`` - 2-digit hexadecimal, zero-padded
 - ``%08h`` - 8-digit hexadecimal, zero-padded
 - ``%0d`` - Decimal, no padding

5. System-Level Verification (8/24)

- ``%s`` - String
- Verbosity Levels: UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL
- Message IDs: Use consistent IDs for filtering and debugging
- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging

5. System-Level Verification (9/24)

- Method Signature:
- ``static function uvm_root get();`` → returns singleton root
- ``function void print_topology();`` → prints hierarchy
- Purpose: Visualize component hierarchy
- Timing: Call in ``end_of_elaboration_phase()`` (after all connections)
- Output: Prints complete component tree with hierarchical paths

5. System-Level Verification (10/24)

- Usage:
- Phase Type: Build-time phase (function, executes bottom-up)
- Purpose: Final elaboration tasks after all connections
- Timing: After ``connect_phase()`, before ``start_of_simulation_phase()`
- Common Uses:
- Print topology

5. System-Level Verification (11/24)

- Validate configuration
- Final checks
- Setup verification
- Pattern: ``forever #period clk = ~clk;``
- Purpose: Generate continuous clock signal
- Period: ``#period`` determines clock period (half period per edge)

5. System-Level Verification (12/24)

- Key: ``forever`` loop runs indefinitely
- Pattern: ``repeat(n) @(posedge clk);``
- Purpose: Wait for n clock cycles
- Usage: Reset sequences, initialization delays
- Key: ``repeat`` loop with event control
- Pattern: Set ConfigDB before ``run_test()``

5. System-Level Verification (13/24)

- Purpose: Configure components before test starts
- Key: Use ``null`` context and ``"*"`` wildcard for global configuration
- Operator: ``@(event)`` - event control
- Purpose: Wait for event to occur
- Behavior: Blocks until event occurs
- Usage: Clock synchronization, signal change detection

5. System-Level Verification (14/24)

- Operator: ``#time`` - delay operator
- Purpose: Wait for specified time
- Behavior: Blocks for specified time units
- Usage: Clock generation, timing delays
- Operator: ``<=`` - non-blocking assignment
- Purpose: Schedule assignment for next simulation time step

5. System-Level Verification (15/24)

- Behavior: All non-blocking assignments in current time step execute together
- Usage: Clock-synchronized signal assignments
- Statement: ``repeat(n) statement;``
- Purpose: Execute statement n times
- Behavior: Loops n times
- Usage: Reset sequences, initialization delays

5. System-Level Verification (16/24)

- Statement: ``forever statement;``
- Purpose: Execute statement indefinitely
- Behavior: Infinite loop
- Usage: Clock generation, continuous monitoring
- Function: ``$sformatf(format, args...)`` → string
- Purpose: Format string with variables

5. System-Level Verification (17/24)

- Return: Formatted string
- Usage: Logging, debugging, reporting
- Problem: Simulation hangs waiting for ``dma_done``
- Solution: Check DUT implementation, add timeout
- Code:
- Prevention: Add bounded wait with timeout checking

5. System-Level Verification (18/24)

- Problem: Simulation hangs waiting for ``tx_busy``
deassert
- Solution: Check DUT implementation, verify clock
- Code:
- Prevention: Add bounded wait with timeout checking
- Problem: ``print_topology()`` shows nothing
- Solution: Call in ``end_of_elaboration_phase()``, not earlier

5. System-Level Verification (19/24)

- Code: Must be after ``connect_phase()`` completes
- Check: Verify all components created in ``build_phase()``
- Problem: VIP agent hardcoded for specific use
- Solution: Use ConfigDB for configuration, avoid hardcoded values
- Code:
- Problem: Too many log messages

5. System-Level Verification (20/24)

- Solution: Use appropriate verbosity levels
- Code: ``UVM_MEDIUM`` for normal, ``UVM_HIGH`` for detailed
- Control: Set verbosity via
``+UVM_VERBOSITY=UVM_MEDIUM``
- Problem: Simulation hangs, no progress
- Solution: Verify clock generation in module
- Code:

5. System-Level Verification (21/24)

- Check: Clock must be in ``initial`` block, not ``always`` block
- Problem: ``uvm_fatal("NO_VIF", ...)`` error
- Solution: Set interface before ``run_test()``
- Code:
- Timing: ConfigDB set → `run_test()` → component `build_phase()` retrieves
- Problem: DUT stuck in reset

5. System-Level Verification (22/24)

- Solution: Verify reset sequence completes
- Code:
- Check: DUT `dma\_done` signal implementation
- Check: Clock running correctly
- Check: Reset sequence completed
- Check: Address/length values valid

5. System-Level Verification (23/24)

- Check: DUT ``tx_busy`` signal implementation
- Check: Clock running correctly
- Check: Loopback connection (if used)
- Check: Data values valid
- Check: Called in ``end_of_elaboration_phase()``
- Check: All components created in ``build_phase()``

5. System-Level Verification (24/24)

- Check: All connections made in ``connect_phase()`
- Check: Virtual interface set via ConfigDB
- Check: Agent configuration retrieved correctly
- Check: Sequencer/driver connections made

Run: Example 7.1: DMA Verification

```
./scripts/module7.sh --dma
cd module7/examples/dma
verilator -sv --cc --exe --timing --trace dma.sv dma.cpp \
-I"$UVM_HOME" +incdir+"$UVM_HOME" +define+UVM_NO_DPI \
"$UVM_HOME/uvm_pkg.sv" ../dut/dma/simple_dma.v --top-module dma
make -C obj_dir -f Vdma.mk
./obj_dir/Vdma
```

Run: Example 7.2: UART Verification

```
./scripts/module7.sh --protocols
cd module7/examples/protocols
verilator -sv --cc --exe --timing --trace uart_example.sv
uart_example.cpp \
-I"$UVM_HOME" +incdir+"$UVM_HOME" +define+UVM_NO_DPI \
"$UVM_HOME/uvm_pkg.sv" ../dut/protocols/uart.v --top-module
uart_example
make -C obj_dir -f Vuart_example.mk
./obj_dir/Vuart_example
```

Run: Example 7.5: VIP Development

```
./scripts/module7.sh --vip  
cd module7/examples/vip  
verilator -sv --cc --exe --timing --trace vip.sv vip.cpp \  
-I"$UVM_HOME" +incdir+"$UVM_HOME" +define+UVM_NO_DPI \  
"$UVM_HOME/uvm_pkg.sv" --top-module vip  
make -C obj_dir -f Vvip.mk  
./obj_dir/Vvip
```

Run: Example 7.6: Best Practices

```
./scripts/module7.sh --best-practices
cd module7/examples/best_practices
verilator -sv --cc --exe --timing --trace best_practices.sv
best_practices.cpp \
-I"$UVM_HOME" +incdir+"$UVM_HOME" +define+UVM_NO_DPI \
"$UVM_HOME/uvm_pkg.sv" --top-module best_practices
make -C obj_dir -f Vbest_practices.mk
./obj_dir/Vbest_practices
```

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --help
```

```
Module 7 self-check  
(run ./scripts/module7.sh when ready)
```

Example 1: DMA Verification

- Interface (``dma_if``): DMA interface with start, done, addresses, length, and channel signals
- Transaction (``DmaTxn``): DMA transaction with source address, destination address, length, and chann...
- Sequence (``DmaSeq``): DMA sequence generating random DMA transfers
- Driver (``DmaDriver``): DMA driver implementing DMA protocol
- Environment (``DmaEnv``): DMA verification environment

Demo: DMA Verification

```
$ ./scripts/module7.sh --dma
```

Terminal: ex01_dma

```
[0:36m===== [0m
[0:36mModule 7: Real-World Applications [0m
[0:36m===== [0m

[0:34m[2026-05-31 04:29:08] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 04:29:08] Checking prerequisites... [0m
[0:31m[2026-05-31 04:29:08] verilator not found [0m
```

Example 2: UART Verification

- Interface (``uart_if``): UART interface with TX/RX signals and control
- Transaction (``UartTxn``): UART transaction with data field
- Sequence (``UartSeq``): UART sequence generating random UART transfers
- Driver (``UartDriver``): UART driver implementing UART protocol
- Environment (``UartEnv``): UART verification environment

Demo: UART Verification

```
$ ./scripts/module7.sh --protocols
```

Terminal: ex02_protocols

```
[0:36m===== [0m
[0:36mModule 7: Real-World Applications [0m
[0:36m===== [0m

[0:34m[2026-05-31 04:29:08] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 04:29:08] Checking prerequisites... [0m
[0:31m[2026-05-31 04:29:08] verilator not found [0m
```

Example 3: SPI Verification

- Test (`SpiExampleTest`): SPI verification test scaffold
- UVM Pattern: Demonstrates UVM structure for SPI verification

Demo: SPI Verification

```
$ ./scripts/module7.sh --protocols
```

Terminal: ex03_protocols

```
[0:36m===== [0m
[0:36mModule 7: Real-World Applications [0m
[0:36m===== [0m

[0:34m[2026-05-31 04:29:09] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 04:29:09] Checking prerequisites... [0m
[0:31m[2026-05-31 04:29:09] verilator not found [0m
```

Example 4: I2C Verification

- Test (``I2cExampleTest``): I2C verification test scaffold
- UVM Pattern: Demonstrates UVM structure for I2C verification

Demo: I2C Verification

```
$ ./scripts/module7.sh --protocols
```

Terminal: ex04_protocols

```
[0:36m===== [0m
[0:36mModule 7: Real-World Applications [0m
[0:36m===== [0m

[0:34m[2026-05-31 04:29:09] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 04:29:09] Checking prerequisites... [0m
[0:31m[2026-05-31 04:29:09] verilator not found [0m
```

Example 5: VIP Development

- Transaction (``VipTxn``): VIP transaction with payload
- Sequence (``VipSeq``): VIP sequence generating transactions
- Driver (``VipDriver``): VIP driver
- Agent (``VipAgent``): VIP agent containing sequencer and driver
- Test (``VipTest``): VIP test demonstrating VIP usage

Demo: VIP Development

```
$ ./scripts/module7.sh --vip && cd module7/examples/vip && verilator  
-sv --cc --exe --timing --trace vip.sv vip.cpp \ && -I"$UVM_HOME"  
+incdir+"$UVM_HOME" +define+UVM_NO_DPI \
```

Terminal: ex05_vip

```
[0:36m===== [0m  
[0:36mModule 7: Real-World Applications [0m  
[0:36m===== [0m  
[0:34m[2026-05-31 04:29:10] Log file: /home/yongfu/proj/universal-verification-methodology/lear  
[0:34m[2026-05-31 04:29:10] Checking prerequisites... [0m  
[0:31m[2026-05-31 04:29:10] verilator not found [0m
```

Example 6: Best Practices

- Test (``BestPracticesTest``): Test demonstrating best practices
- Phase Organization: Proper phase usage and logging
- Topology Printing: Component hierarchy visualization
- Logging Patterns: Structured logging for maintainability

Demo: Best Practices

```
$ ./scripts/module7.sh --best-practices
```

Terminal: ex06_best_practices

```
[0:36m===== [0m
[0:36mModule 7: Real-World Applications [0m
[0:36m===== [0m

[0:34m[2026-05-31 04:29:10] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 04:29:10] Checking prerequisites... [0m
[0:31m[2026-05-31 04:29:10] verilator not found [0m
```

Common pitfalls

DMA Done Not Detected

- Problem: Simulation hangs waiting for ``dma_done``
- Solution: Check DUT implementation, add timeout
- Code:
- Prevention: Add bounded wait with timeout checking

UART Busy Never Deasserts

- Problem: Simulation hangs waiting for ``tx_busy`` deassert
- Solution: Check DUT implementation, verify clock
- Code:
- Prevention: Add bounded wait with timeout checking

Topology Not Printed

- Problem: ``print_topology()` shows nothing
- Solution: Call in ``end_of_elaboration_phase()`, not earlier
- Code: Must be after ``connect_phase()` completes
- Check: Verify all components created in ``build_phase()`

VIP Agent Not Reusable

- Problem: VIP agent hardcoded for specific use
- Solution: Use ConfigDB for configuration, avoid hardcoded values
- Code:

Phase Logging Too Verbose

- Problem: Too many log messages
- Solution: Use appropriate verbosity levels
- Code: ``UVM_MEDIUM`` for normal, ``UVM_HIGH`` for detailed
- Control: Set verbosity via
``+UVM_VERBOSITY=UVM_MEDIUM``

Clock Not Running

- Problem: Simulation hangs, no progress
- Solution: Verify clock generation in module
- Code:
- Check: Clock must be in ``initial`` block, not ``always`` block

Practice & assessment

What you should know (1/2)

- Verify complex designs (DMA, protocols) using UVM patterns
- Develop reusable VIP with proper architecture
- Apply best practices for code organization and maintainability
- Perform system-level verification with multiple DUTs
- Create production-quality testbenches
- Understand protocol verification patterns

What you should know (2/2)

- Package verification components for reuse
- Use topology printing for debugging
- Implement phase-based logging
- Apply real-world verification patterns

Exercises

- DMA Verification
- Protocol Verification
- VIP Development
- Best Practices
- System-Level Verification

Assessment checklist

- Can verify complex designs (DMA, protocols)
- Can develop reusable VIP
- Can apply best practices
- Can perform system-level verification
- Can create production-quality testbenches
- Understands protocol verification patterns

Summary & next steps

- Apply UVM to real-world verification scenarios
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course